Linux Device Tree

İçindekiler

1. Problem: Kernel Donanımı Nereden Bilecek?
2. Device Tree Nedir?
3. Dosya Türleri: DTS, DTSI, DTC, DTB, DTBO
4. Söz Dizimi (Syntax)
5. Bir Device Tree'nin Genel Yapısı
6. Önemli Property'ler
7. compatible ile Driver Eşleşmesi
8. Kalıtım: .dtsi, include ve Label ile Override
9. dtc Kullanımı
10. Boot Sürecinde Device Tree
11. Çalışan Sistemde Device Tree'yi İncelemek
12. Device Tree Overlay (.dtbo)
13. Özet

---

1. Problem: Kernel Donanımı Nereden Bilecek?

Kernel'in çalışabilmesi için kartta **hangi donanımların olduğunu, nerede durduklarını ve nasıl bağlandıklarını** bilmesi gerekir. Örneğin:

- UART denetleyicisinin register'ları hangi adreste?
- Kaç MB RAM var, hangi adresten başlıyor?
- I2C hattına hangi çip, hangi adresle bağlı?
- USR0 LED'i hangi GPIO pinine bağlı?
- Hangi cihaz hangi interrupt hattını kullanıyor?

Keşfedilebilen ve keşfedilemeyen donanım

	Keşfedilebilir (discoverable)	Keşfedilemez (non-discoverable)
Örnek	PCI, PCIe, USB	UART, I2C, SPI, GPIO, Ethernet MAC, NAND, SoC içindeki çoğu çevre birimi
Nasıl bulunur?	Cihaz kendini tanıtır (Vendor ID / Device ID). Kernel bus'ı tarayarak bulur.	Cihaz kendini tanıtmaz. Kernel'e **birinin söylemesi gerekir**.

Masaüstü PC'lerde (x86) bu bilgi BIOS/UEFI tarafından **ACPI tabloları** ile verilir. Gömülü sistemlerde (ARM, RISC-V, PowerPC) ise donanımın büyük kısmı SoC'un içinde, belirli adreslerde duran ve kendini tanıtmayan bloklardır. Bu yüzden bu bilgiyi kernel'e bir şekilde aktarmak gerekir.

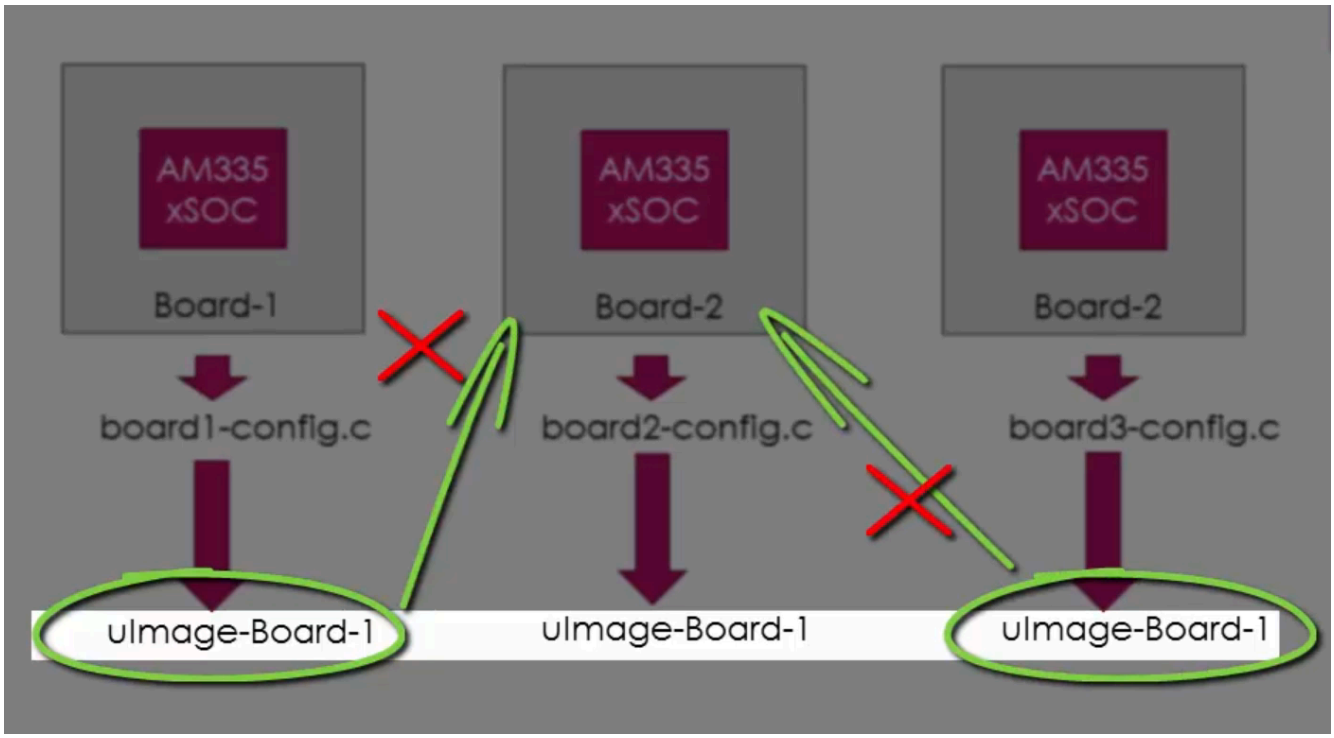
Eski yöntem: board file'lar

Device Tree'den önce ARM'da her kart için kernel kaynak kodunda bir C dosyası (**board file**) yazılırdı (`arch/arm/mach-xxx/board-yyy.c`). Donanım bilgisi bu dosyalara sabit olarak gömülürdü:

```
/* Eski usül: donanım C koduna gömülü (temsili örnek) */
static struct resource uart0_resources[] = {
    { .start = 0x44E09000, .end = 0x44E09FFF, .flags = IORESOURCE_MEM },
    { .start = 72,         .end = 72,         .flags = IORESOURCE_IRQ },
};
```

Bu yöntemin sorunları:

- Her yeni kart = kernel'e yeni C kodu. `arch/arm` altında binlerce kart dosyası birikti.
- Kartta küçük bir değişiklik (ör. LED'in başka pine taşınması) bile **kernel'i yeniden derlemeyi** gerektiriyordu.
- Her kart için ayrı bir kernel image'ı gerekiyordu. **Tek bir kernel, birden fazla kartta çalışamıyordu.**



2011'de Linus Torvalds ARM tarafındaki bu karmaşadan açıkça şikayet etti. Bunun üzerine ARM dünyası, PowerPC'de zaten kullanılan **Device Tree**'ye geçti (kernel 3.x serisi). Bugün yeni ARM32 ve ARM64 kartları için Device Tree zorunludur.

> **Not**
>
> Temel fikir: **Donanım tarifi koddan ayrılır.** Kernel aynı kalır, kartı tarif eden küçük bir veri dosyası (DTB) değişir. Böylece aynı `zImage` hem BeagleBone Black'te hem de başka bir AM335x kartında çalışabilir. Sadece DTB farklıdır.

---

2. Device Tree Nedir?

Device Tree (DT), bir sistemdeki donanımı tarif eden, **ağaç (tree) şeklinde bir veri yapısıdır**.

- Ağaç **node**'lardan (düğüm) oluşur. Her node bir donanım bloğunu temsil eder (CPU, bellek, UART, I2C denetleyicisi, EEPROM...).
- Her node'un **property**'leri (özellikleri) vardır: adres, interrupt numarası, uyumlu driver adı, durum vb.
- Node'lar iç içe olabilir. Bir I2C denetleyicisinin altında ona bağlı I2C cihazları alt node olarak yer alır. Bu, donanımdaki **fiziksel bağlantı hiyerarşisini** yansıtır.
- Kökeni Sun'ın **OpenFirmware** standardına dayanır. Bugün **devicetree.org** tarafından Devicetree Specification olarak standartlaştırılmıştır.
- **İşletim sisteminden bağımsızdır**: Linux, U-Boot, FreeBSD, Zephyr gibi projeler de kullanır.

> **Önemli**
>
> Device Tree **donanımın ne olduğunu** tarif eder, **nasıl kullanılacağını** değil. "Bu kartta 0x44E09000 adresinde bir UART var" bir DT bilgisidir. "Bu UART'ı 115200 baud ile konsol olarak kullan" ise bir yapılandırma tercihidir ve genelde kernel command line'a (`bootargs`) aittir.

Device Tree'de olmayan bir şey kernel tarafından **bilinemez**. Driver kernel'e derlenmiş olsa bile, DT'de ona uyan bir node yoksa o driver hiç çalışmaz (probe edilmez).

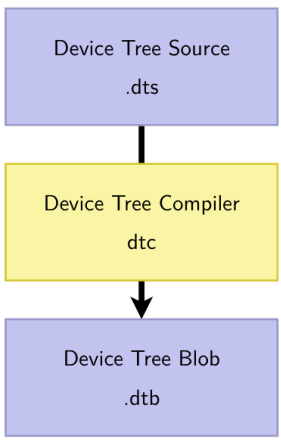
---

3. Dosya Türleri: DTS, DTSI, DTC, DTB, DTBO



Device Tree: from source to blob

- A tree data structure describing the hardware is written by a developer in a **Device Tree Source** file, `.dts`
- Processed by the **Device Tree Compiler**, `dtc`
- Produces a more efficient representation: **Device Tree Blob**, `.dtb`
- Additional C preprocessor pass
- `.dtb` → accurately describes the hardware platform in an **OS-agnostic** way.
- `.dtb`≈ few dozens of kilobytes
- DTB also called **FDT**, *Flattened Device Tree*, once loaded into memory.
  - `fdt` command in U-Boot
  - `fdt_` APIs



Görsel: Bootlin, "Linux kernel and driver development" eğitim slaytları (CC BY-SA 3.0)

Kısaltma	Açılımı	Ne?
DT	Device Tree	Genel kavram: donanımı tarif eden ağaç yapısı
.dts	Device Tree **Source**	İnsanın okuyup yazabildiği **metin** kaynak dosyası. Bir kartı tarif eder. Örn: `am335x-boneblack.dts`
.dtsi	Device Tree Source **Include**	Başka dosyalara `#include` edilen ortak parça. Genelde SoC'u ya da bir kart ailesini tarif eder. Örn: `am33xx.dtsi`. C'deki `.h` dosyalarına benzer.
dtc	Device Tree **Compiler**	`.dts` dosyasını `.dtb`'ye derleyen (ve tersini yapabilen) araç
.dtb	Device Tree **Blob** (Binary)	Derlenmiş, **ikili (binary)** çıktı. Bootloader bunu RAM'e yükler ve kernel bunu okur. Örn: `am335x-boneblack.dtb`
FDT	Flattened Device Tree	DTB'nin bellekteki "düzleştirilmiş" formatının adı. U-Boot'taki `fdt` komutu ve `fdtaddr` değişkeni buradan gelir.
.dtbo	Device Tree Blob **Overlay**	Var olan bir DTB'ye çalışma/boot anında eklenen **yama**. Cape/HAT gibi eklenti kartlar için kullanılır. (Bölüm 12)

Derleme akışı

```
am33xx.dtsi ─┐
am335x-bone-common.dtsi ─┤  #include
am335x-boneblack-common.dtsi ─┤
                              ▼
            am335x-boneblack.dts
                    │
                    │  1) C preprocessor (cpp): #include, #define çözülür
                    │  2) dtc: metin → binary
                    ▼
            am335x-boneblack.dtb   (~60-70 KB)
                    │
                    │  U-Boot RAM'e yükler (fdtaddr)
                    ▼
                Linux kernel
```

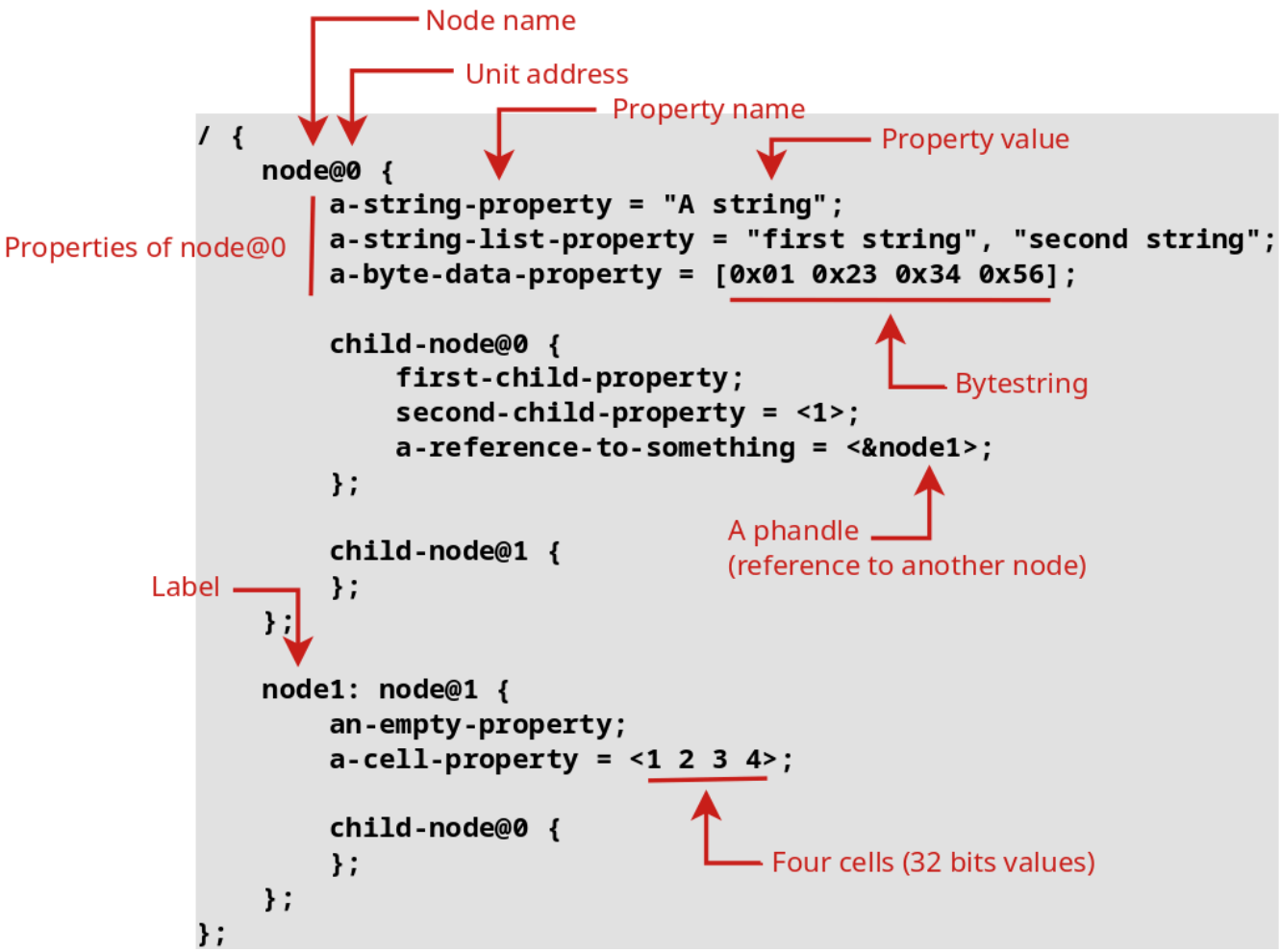
- Kernel kaynak ağacında `.dts` dosyaları önce **C ön işlemcisinden (cpp)** geçer. Bu sayede `#include <dt-bindings/gpio/gpio.h>` ve `GPIO_ACTIVE_HIGH` gibi sabitler kullanılabilir. Sonra `dtc` ile derlenir.
- DTB küçüktür (birkaç on KB) ve kernel'in hızlıca okuyabileceği bir formattadır.
- Her DTB dosyası `0xd00dfeed` sihirli sayısı (magic number) ile başlar. Bir dosyanın gerçekten DTB olup olmadığını böyle anlayabilirsin:

```
$ hexdump -C am335x-boneblack.dtb | head -1     # ilk 4 byte: d0 0d fe ed
00000000  d0 0d fe ed 00 01 0a 5c  00 00 00 38 00 00 f8 a8  |.......\...8....|
```

> **Not**
>
> Kernel tarafındaki `.dts` dosyaları kernel kaynak kodunun içindedir: ARM32 için `arch/arm/boot/dts/`, ARM64 için `arch/arm64/boot/dts/<vendor>/`. Kernel 6.5'ten itibaren ARM32 dosyaları da vendor klasörlerine taşındı. BeagleBone Black'in dosyası yeni kernel'lerde `arch/arm/boot/dts/ti/omap/am335x-boneblack.dts` konumundadır.

---

4. Söz Dizimi (Syntax)



Görsel: Bootlin eğitim materyalleri (CC BY-SA 3.0)

Node

```
[label:] node-name[@unit-address] {
    [properties]
    [child nodes]
};
```

Parça	Açıklama	Örnek
`/`	Kök (root) node. Her DT'de bir tane vardır ve tüm sistemi temsil eder.	`/ { ... };`
node-name	Cihazın **türünü** anlatan genel bir isim	`serial`, `i2c`, `gpio`, `memory`, `eeprom`
@unit-address	Node'un adresi. Genelde `reg` property'sinin ilk değeriyle aynıdır.	`serial@44e09000`, `eeprom@50`
label	Node'a verilen takma ad. Başka yerlerden `&label` diye referans verilir.	`uart0: serial@44e09000`
`;`	Her node ve property `;` ile biter. Unutmak en sık yapılan derleme hatasıdır.	

Bir node'un tam yolu (path) dosya sistemi yoluna benzer: `/ocp/interconnect@44c00000/.../serial@0`.

Property

Property'ler `isim = değer;` şeklindedir. Değer türleri:

Tür	Yazım	Örnek
Boş (boolean)	Sadece isim. Varsa "true", yoksa "false" demektir.	`gpio-controller;`
String	Çift tırnak	`status = "okay";`
String listesi	Virgülle ayrılmış string'ler	`compatible = "ti,am3352-uart", "ti,omap3-uart";`
Cell (u32) listesi	`< >` içinde 32-bit sayılar	`reg = <0x44e09000 0x2000>;`
Byte dizisi	`[ ]` içinde hex byte'lar	`local-mac-address = [00 11 22 33 44 55];`
phandle (referans)	`< >` içinde `&label`	`gpios = <&gpio1 21 GPIO_ACTIVE_HIGH>;`

> İpucu
>
> **phandle**, bir node'un başka bir node'a "işaretçi" (pointer) gibi referans vermesidir. Yukarıdaki `gpios` örneği şunu söyler: "Bu LED, `gpio1` denetleyicisinin **21.** pinine bağlı ve **active-high**." dtc derleme sırasında `&gpio1` 'i o node'a ait benzersiz bir sayıya (phandle) çevirir.

---

5. Bir Device Tree'nin Genel Yapısı

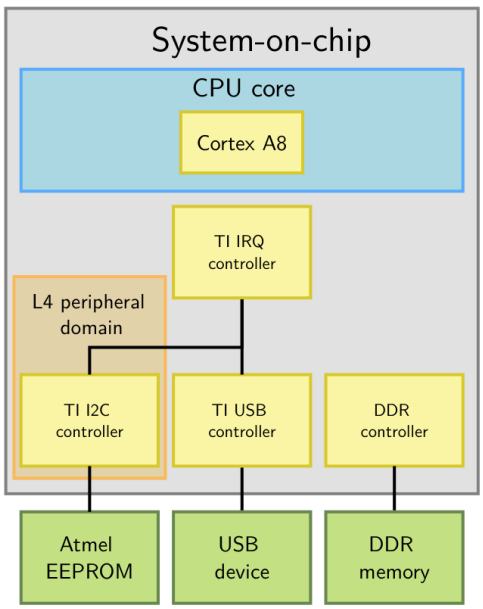


DT overall structure: simplified example

```
/ {
  cpus { ... };
  memory@0x80000000 { ... };
  chosen { ... };
  ocp {
    intc: interrupt-controller@48200000 { ... };
    usb0: usb@47401300 { ... };
    l4_per: interconnect@44c00000 {

      i2c0: i2c@40012000 {
        compatible = "ti,omap4-i2c";
        #address-cells = <1>;
        #size-cells = <0>;
        reg = <0x0 0x1000>;
        interrupts = <70>;
        status = "okay";
        pinctrl-names = "default";
        pinctrl-0 = <&i2c0_pins>;
        clock-frequency = <400000>;

        baseboard_eeprom: eeprom@50 {
          compatible = "atmel,24c256";
          reg = <0x50>;
        };
      };
    };
  };
};
```



Görsel: Bootlin eğitim slaytları (CC BY-SA 3.0). Soldaki ağaç, sağdaki AM335x tabanlı donanımı tarif ediyor.

Görseldeki eşleşmeye dikkat et: DT'deki iç içe yapı **donanımdaki bağlantıyı** birebir yansıtır. EEPROM, I2C denetleyicisine bağlı olduğu için DT'de de `i2c@40012000` node'unun **altındadır**.

Hemen her DT'de bulunan standart node'lar:

Node	Görevi
`/`	Kök. `model` (kartın adı) ve `compatible` (kartın kimliği) burada tanımlanır.
`cpus`	İşlemci çekirdekleri (BBB'de tek bir Cortex-A8)
`memory@...`	RAM'in başlangıç adresi ve boyutu
`chosen`	Donanım değil, bootloader ile kernel arasındaki bilgiler: `bootargs`, `stdout-path` (konsol)
`aliases`	Kısa isimler. Örn: `serial0 = &uart0;` sayesinde `uart0` → `/dev/ttyS0` olur.
`ocp` / `soc`	SoC içindeki çevre birimlerinin bulunduğu bus. AM335x'te adı `ocp`'dir (On-Chip Peripherals).

BeagleBone Black'ten gerçek bir örnek

BBB'nin ana DTS dosyası aslında çok kısadır. Asıl iş include ettiği dosyalardadır:

```
/* am335x-boneblack.dts (kısaltılmış) */
/dts-v1/;

#include "am33xx.dtsi"                    /* AM335x SoC tanımı */
#include "am335x-bone-common.dtsi"        /* Tüm BeagleBone'larda ortak */
#include "am335x-boneblack-common.dtsi"   /* BBB ve BBB Wireless'ta ortak */
#include "am335x-boneblack-hdmi.dtsi"     /* HDMI desteği */

/ {
    model = "TI AM335x BeagleBone Black";
    compatible = "ti,am335x-bone-black", "ti,am335x-bone", "ti,am33xx";
};
```

`am335x-bone-common.dtsi` içinden USR LED tanımı (kısaltılmış):

```
leds {
    pinctrl-names = "default";
    pinctrl-0 = <&user_leds_s0>;      /* pin mux ayarı */
    compatible = "gpio-leds";         /* hangi driver? → leds-gpio */

    led2 {
        label = "beaglebone:green:usr0";
        gpios = <&gpio1 21 GPIO_ACTIVE_HIGH>;    /* GPIO1_21 pini */
        linux,default-trigger = "heartbeat";    /* kalp atışı gibi yanıp söner */
        default-state = "off";
    };
    /* usr1, usr2, usr3 ... */
};
```

> **İpucu**
> Kart açıldığında USR0 LED'inin "kalp atışı" gibi yanıp sönmesinin sebebi tam olarak bu satırdır: `linux,default-trigger = "heartbeat";`. Yani bu davranış kernel kodunda değil, **Device Tree'de** tanımlıdır.

---

6. Önemli Property'ler

Property	Anlamı	Örnek
`compatible`	**En önemli property.** Cihazın ne olduğunu ve hangi driver'ın onu yöneteceğini belirler. Format: `"üretici,model"`. En özelden en genele doğru sıralanır.	`"ti,am3352-uart", "ti,omap3-uart"`
`reg`	Cihazın adresi. Memory-mapped cihazlarda **(başlangıç adresi, boyut)** çifti. I2C/SPI cihazlarında bus üzerindeki adres.	`reg = <0x44e09000 0x2000>;` `reg = <0x50>;`
`#address-cells`	Alt node'ların `reg` değerindeki **adresin** kaç adet 32-bit cell olduğu	`<1>`
`#size-cells`	Alt node'ların `reg` değerindeki **boyutun** kaç cell olduğu. I2C'de boyut yoktur, bu yüzden `<0>`.	`<1>` veya `<0>`
`status`	Cihaz kullanılıyor mu? `"okay"` → evet, `"disabled"` → hayır (driver probe edilmez).	`status = "okay";`
`interrupts`	Cihazın kullandığı interrupt hattı	`interrupts = <72>;`
`interrupt-parent`	Interrupt'ın hangi interrupt denetleyicisine gittiği (phandle)	`<&intc>`
`clocks`	Cihazın beslendiği saat (clock) kaynakları	`clocks = <&l4ls_gclk>;`
`pinctrl-0`, `pinctrl-names`	Pin multiplexing: SoC pinlerinin bu cihaz için hangi moda alınacağı	`pinctrl-0 = <&uart0_pins>;`
`gpios`, `*-gpios`	Kullanılan GPIO pinleri (phandle + pin no + flag)	`<&gpio1 21 GPIO_ACTIVE_HIGH>`

reg, #address-cells ve #size-cells ilişkisi

`reg`'in nasıl okunacağını **parent node'daki** `#address-cells` ve `#size-cells` belirler:

```
ocp {
    #address-cells = <1>;   /* alt node'larda adres = 1 cell */
    #size-cells = <1>;      /* alt node'larda boyut = 1 cell */

    serial@44e09000 {
        reg = <0x44e09000 0x2000>;   /* adres=0x44E09000, boyut=8 KB */
    };
};

i2c0: i2c@44e0b000 {
    #address-cells = <1>;
    #size-cells = <0>;      /* I2C cihazlarının "boyutu" yok */

    baseboard_eeprom: eeprom@50 {
        compatible = "atmel,24c256";
        reg = <0x50>;       /* I2C adresi 0x50 */
    };
};
```

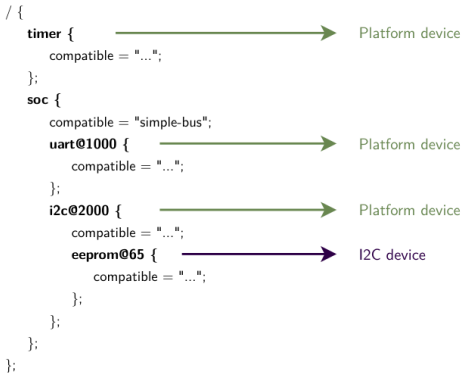
> **Not**
>
> BBB'nin üzerindeki **EEPROM** (kart adını ve seri numarasını tutan çip) gerçekten de I2C0 hattında **0x50** adresindedir. U-Boot bu EEPROM'u okuyarak kartın BeagleBone Black olduğunu anlar ve doğru DTB'yi (`am335x-boneblack.dtb`) seçer.

---

7. compatible ile Driver Eşleşmesi

Kernel DTB'yi okuduğunda her node için bir **device** oluşturur. Sonra her device için `compatible` string'ine bakar ve **aynı string'i destekleyen bir driver** arar. Eşleşme bulunursa driver'ın `probe()` fonksiyonu çağrılır ve cihaz o driver'a **bağlanır (bind)**.

- Linux identifies as **platform devices**:
  - Top-level DT nodes with a `compatible` string
  - Sub-nodes of `simple-bus`
    - Instantiated automatically at boot time
- Sub-nodes of I2C controllers → *I2C devices*
- Sub-nodes of SPI controllers → *SPI devices*
- Each Linux driver has a table of compatible strings it supports
  - struct of_device_id[]
- When a DT node compatible string matches a given driver, the device is *bound* to that driver.



Görsel: Bootlin eğitim slaytları (CC BY-SA 3.0)

- Kök node'un hemen altındaki ve `simple-bus` altındaki node'lar → **platform device** olur.
- I2C denetleyicisinin altındaki node'lar → **I2C device** olur.
- SPI denetleyicisinin altındaki node'lar → **SPI device** olur.

Driver tarafında bu eşleşme `of_device_id` tablosu ile yapılır (`of` = OpenFirmware):

```
/* drivers/i2c/busses/i2c-omap.c (kısaltılmış) */
static const struct of_device_id omap_i2c_of_match[] = {
    { .compatible = "ti,omap4-i2c", .data = &omap4_pdata },
    { .compatible = "ti,omap3-i2c", .data = &omap3_pdata },
    { },
};
MODULE_DEVICE_TABLE(of, omap_i2c_of_match);

static struct platform_driver omap_i2c_driver = {
    .probe  = omap_i2c_probe,
    .remove = omap_i2c_remove,
    .driver = {
        .name           = "omap_i2c",
        .of_match_table = omap_i2c_of_match,   /* DT eşleşme tablosu */
    },
};
```

```
DT node:   compatible = "ti,omap4-i2c"
                    |
                    |  string eşleşmesi
                    ▼
Driver:    { .compatible = "ti,omap4-i2c" }  →  omap_i2c_probe() çağrılır
```

Driver `probe()` içinde `reg`, `interrupts`, `clocks` gibi property'leri okuyarak cihazı başlatır (`platform_get_resource()`, `of_property_read_u32()` vb.).

> **İpucu**
>
> `compatible` listesinde birden fazla string olmasının sebebi **geriye dönük uyumluluktur**. `"ti,am3352-uart", "ti,omap3-uart"` şunu söyler: "Ben aslında AM3352 UART'ıyım. Bana özel driver yoksa OMAP3 UART driver'ı da beni sürebilir." Kernel listeyi soldan sağa dener.

> **Not**
>
> Bir cihazın property'lerinin nasıl yazılacağı (hangi property zorunlu, hangi değerler geçerli) **binding** denen belgelerle tanımlanır. Bunlar kernel'de `Documentation/devicetree/bindings/` altındadır ve günümüzde YAML formatındadır. Yeni bir cihaz eklerken ilk bakılacak yer burasıdır.

---

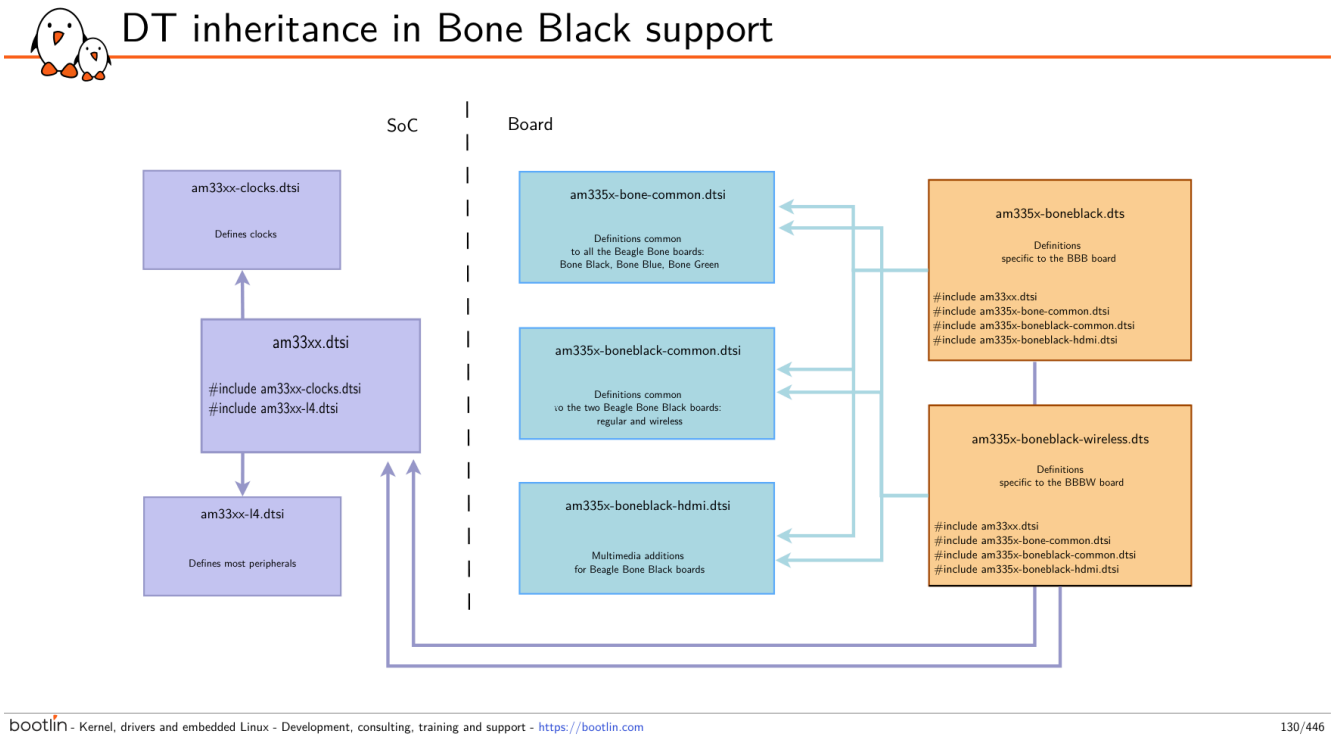
8. Kalıtım: .dtsi, include ve Label ile Override

Aynı SoC'u kullanan onlarca kart vardır. Her kart için SoC'un tamamını baştan yazmak yerine:

1. SoC'un tüm çevre birimleri bir `.dtsi` dosyasında tanımlanır. Çoğu `status = "disabled"` olarak bırakılır.
2. Kartın `.dts` dosyası bu `.dtsi`'yi include eder. Kartta kullanılan birimleri `status = "okay"` yaparak açar ve karta özel cihazları (EEPROM, LED, PMIC...) ekler.

Aynı node birden fazla kez tanımlanırsa dtc bunları **birleştirir**. Aynı property iki yerde varsa **sonra gelen kazanır**.

BeagleBone Black'in include zinciri



Görsel: Bootlin eğitim slaytları (CC BY-SA 3.0)

Dosya	Seviye	İçerik
`am33xx.dtsi`	SoC	AM335x işlemcinin genel tanımı. `am33xx-l4.dtsi` (çevre birimleri) ve `am33xx-clocks.dtsi` (saatler) dosyalarını include eder.
`am335x-bone-common.dtsi`	Kart ailesi	Tüm BeagleBone'larda ortak olanlar: LED'ler, EEPROM, PMIC (TPS65217), 256 MB RAM
`am335x-boneblack-common.dtsi`	Kart ailesi	BBB ve BBB Wireless'ta ortak olanlar: eMMC, 512 MB RAM
`am335x-boneblack-hdmi.dtsi`	Ek özellik	HDMI çıkışı (TDA19988 çipi)
`am335x-boneblack.dts`	Kart	Sadece BBB'ye özel olanlar: `model`, `compatible`

Override'a güzel bir örnek RAM boyutudur. `am335x-bone-common.dtsi` RAM'i 256 MB tanımlar (eski BeagleBone White'ta 256 MB vardı). `am335x-boneblack-common.dtsi` aynı node'u tekrar yazarak bunu 512 MB'a çıkarır:

```
/* am335x-bone-common.dtsi */
memory@80000000 {
    device_type = "memory";
    reg = <0x80000000 0x10000000>;   /* 256 MB */
};

/* am335x-boneblack-common.dtsi (sonra include edildiği için kazanır) */
memory@80000000 {
    device_type = "memory";
    reg = <0x80000000 0x20000000>;   /* 512 MB */
};
```

Label ile override (tercih edilen yöntem)

Bir node'u değiştirmek için tam yolunu tekrar yazmak yerine, **label**'ı ile doğrudan ona erişmek tercih edilir:



Inheritance and labels

Doing:

soc.dtsi

```
/ {
  ocp {
    uart0: serial@0 {
      compatible = "ti,am3352-uart";
      reg = <0x0 0x1000>;
      status = "disabled";
    };
  };
};
```

board.dts

```
#include "soc.dtsi"

/ {
  ocp {
    serial@0 {
      status = "okay";
    };
  };
};
```

Is exactly equivalent to:

soc.dtsi

```
/ {
  ocp {
    uart0: serial@0 {
      compatible = "ti,am3352-uart";
      reg = <0x0 0x1000>;
      status = "disabled";
    };
  };
};
```

board.dts

```
#include "soc.dtsi"

&uart0 {
  status = "okay";
};
```

→ this solution is preferred

bootlin - Kernel, drivers and embedded Linux - Development, consulting, training and support - https://bootlin.com 129/446

Görsel: Bootlin eğitim slaytları (CC BY-SA 3.0). Soldaki ve sağdaki board.dts tamamen aynı sonucu üretir. Sağdaki `&uart0 { ... };` yazımı tercih edilir.

```
/* Kart dosyasında: SoC'ta "disabled" olan I2C2'yi aç ve hızını ayarla */
&i2c2 {
    status = "okay";
    clock-frequency = <100000>;
};
```

9. dtc Kullanımı



dtc example

```
$ cat foo.dts
/dts-v1/;

/ {
        welcome = <0xBADCAFE>;
        bootlin {
                webinar = "great";
                demo = <1>, <2>, <3>;
        };
};
```

```
$ dtc -I dtb -O dts foo.dtb
/dts-v1/;

/ {
        welcome = <0xbadcafe>;

        bootlin {
                webinar = "great";
                demo = <0x01 0x02 0x03>;
        };
};
```

```
$ dtc -I dts -O dtb -o foo.dtb foo.dts
$ ls -l foo.dt*
-rw-r--r-- 1 thomas thomas 169 ... foo.dtb
-rw-r--r-- 1 thomas thomas 102 ... foo.dts
```


Görsel: Bootlin eğitim slaytları (CC BY-SA 3.0)

Kurulum

```
sudo apt install device-tree-compiler
```

Temel komutlar

```
# DTS → DTB (derleme)
dtc -I dts -O dtb -o foo.dtb foo.dts

# DTB → DTS (geri derleme / decompile)
dtc -I dtb -O dts -o foo_geri.dts foo.dtb

# Çalışan sistemin canlı device tree'sini DTS olarak dök
dtc -I fs -O dts /proc/device-tree
```

Parametre	Anlamı
`-I`	Girdi formatı: `dts`, `dtb` veya `fs` (dosya sistemi, ör. `/proc/device-tree`)
`-O`	Çıktı formatı: `dts`, `dtb` veya `asm`
`-o`	Çıktı dosyası
`-@`	Label'ları (`__symbols__`) DTB'ye ekle. Overlay kullanılacaksa gereklidir.

> **İpucu**
> Geri derleme (`-I dtb -O dts`) çok işe yarar: Elinde sadece bir `.dtb` varsa (ör. hazır bir image'dan çıkan) içinde ne olduğunu görebilirsin. Ancak çıkan DTS'te yorumlar, `#include`'lar, `#define` isimleri ve label'lar kaybolmuştur. Sayılar ham hex olarak görünür (`GPIO_ACTIVE_HIGH` yerine `0x00` gibi).

Kernel kaynak ağacında DTB derleme

Kernel'in içindeki `.dts` dosyaları elle `dtc` çağırmak yerine kernel'in build sistemiyle derlenir. Bu yöntem cpp ön işleme adımını da otomatik yapar:

```
# Tüm DTB'leri derle (kernel kaynak klasöründe)
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- dtbs

# Sadece BBB'nin DTB'sini derle (kernel 6.5+ yol yapısı)
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- ti/omap/am335x-boneblack.dtb

# Çıktı:
#   arch/arm/boot/dts/ti/omap/am335x-boneblack.dtb
```

> **Not**
> Kernel 6.5'ten eski sürümlerde hedef sadece `am335x-boneblack.dtb`'dir ve çıktı `arch/arm/boot/dts/am335x-boneblack.dtb` konumuna yazılır.

---

10. Boot Sürecinde Device Tree

Linux boot sürecinde gördüğümüz gibi U-Boot, kernel'i başlatmadan önce **hem kernel image'ını hem de DTB'yi** RAM'e yükler:

```
flowchart LR
    A["eMMC / SD kart<br/>/boot/zImage<br/>/boot/dtbs/am335x-boneblack.dtb"] -->|"U-Boot: load"| B["DDR RAM<br/>loadaddr = 0x82000000 → kernel<br/>fdtaddr = 0x88000000 → DTB"]
    B -->|"bootz / bootm<br/>r2 = DTB adresi"| C["Linux kernel<br/>DTB'yi okur (unflatten)<br/>device'ları oluşturur"]
    C --> D["Driver'lar compatible ile<br/>eşleşir ve probe edilir"]
```

SD karttan boot bölümündeki `uEnv.txt` ile birebir aynı mantık:

```
fdtaddr=0x88000000                                  # DTB'nin RAM'deki adresi
load mmc 0:2 ${fdtaddr} /boot/am335x-boneblack.dtb  # DTB'yi RAM'e yükle
bootm ${loadaddr} - ${fdtaddr}                      # kernel'i başlat ve DTB adresini ver
```

`bootm` / `bootz` komutunun üç parametresi:

Parametre	Anlamı
`${loadaddr}`	Kernel image'ının RAM adresi
`-`	initramfs adresi (`-` = yok)
`${fdtaddr}`	**DTB'nin RAM adresi**

ARM32'de U-Boot kernel'e atlarken DTB'nin fiziksel adresini `r2` **register'ında** verir. Kernel açılışta bu adresteki blob'u okur, `0xd00dfeed` magic number'ını kontrol eder ve DTB'yi bellekte gezilebilir bir ağaca dönüştürür (**unflatten**).

> **Not**
> U-Boot, kernel'e vermeden önce DTB'yi **değiştirebilir** de. Örneğin `bootargs` değişkenini DTB'deki `/chosen` node'una yazar ve RAM boyutunu `/memory` node'una işler. Kernel command line'ının kernel'e ulaşma yollarından biri budur.

U-Boot'ta DTB'yi incelemek

U-Boot'un `fdt` komutuyla RAM'deki DTB'ye bakılabilir:

```
=> load mmc 0:2 ${fdtaddr} /boot/am335x-boneblack.dtb
=> fdt addr ${fdtaddr}           # hangi adresteki DTB ile çalışılacak
=> fdt print /                   # tüm ağacı yazdır
=> fdt print /leds               # sadece leds node'u
=> fdt list /ocp                 # sadece alt node isimleri
```

---

11. Çalışan Sistemde Device Tree'yi İncelemek

Kernel, okuduğu device tree'yi bir dosya sistemi olarak dışarı açar:

```
/sys/firmware/devicetree/base/     # asıl konum
/proc/device-tree                  # → yukarıdakine sembolik link
```

Burada **her node bir klasör, her property bir dosyadır**:

```
# Kartın modeli
$ cat /proc/device-tree/model
TI AM335x BeagleBone Black

# Kök node'un compatible listesi (string'ler \0 ile ayrılmıştır)
$ cat /proc/device-tree/compatible | tr '\0' '\n'
ti,am335x-bone-black
ti,am335x-bone
ti,am33xx

# LED node'larına bak
$ ls /proc/device-tree/leds/

# Bootloader'ın kernel'e verdiği bootargs
$ cat /proc/device-tree/chosen/bootargs

# Sayısal property'leri hex olarak gör (ör. RAM adresi ve boyutu)
$ hexdump -C /proc/device-tree/memory@80000000/reg
00000000  80 00 00 00 20 00 00 00                           |.... ...|
#         adres=0x80000000  boyut=0x20000000 (512 MB)
```

Tüm ağacı tek seferde okunabilir DTS olarak görmek için:

```
sudo apt install device-tree-compiler
dtc -I fs -O dts /proc/device-tree > canli.dts 2>/dev/null
less canli.dts
```

Bir cihazın hangi driver'a bağlandığını görmek için:

```
ls /sys/bus/platform/devices/          # DT'den oluşturulan platform device'lar
ls -l /sys/bus/platform/devices/44e09000.serial/driver
#  → .../drivers/omap8250   (UART0'ı süren driver, kernel config'e göre serial-omap da
olabilir)
```

BBB'nin Debian image'larında DTB dosyaları `/boot/dtbs/<kernel-sürümü>/` klasöründedir:

```
ls /boot/dtbs/$(uname -r)/ | grep boneblack
```

> **İpucu**
> Bir donanım çalışmıyorsa ilk kontrol: `/proc/device-tree` altında o cihazın node'u var mı ve `status` değeri `okay` mi? Node yoksa ya da `disabled` ise driver hiç devreye girmez. `dmesg | grep -i <cihaz>` ile de probe hatalarına bakılabilir.

---

12. Device Tree Overlay (.dtbo)

BeagleBone'a **cape** (P8/P9 header'larına takılan eklenti kart) takıldığında donanım değişir. Her cape kombinasyonu için ayrı bir `.dts` yazmak mantıklı değildir. Bunun çözümü **overlay**'dir.

- Overlay, ana DTB'ye **sonradan uygulanan bir yamadır**. Node ekleyebilir veya mevcut node'ları değiştirebilir.
- `.dts` kaynağından `dtc -@` ile `.dtbo` olarak derlenir.
- Ana DTB'nin de `-@` ile derlenmiş olması gerekir (label'ların bulunabilmesi için).
- Genellikle U-Boot tarafından boot sırasında uygulanır.

Basit bir overlay örneği (P9 header'ındaki I2C2'yi açar):

```
/dts-v1/;
/plugin/;                 /* bu bir overlay */

&i2c2 {                   /* ana DTB'deki i2c2 label'ına yama */
    status = "okay";
    clock-frequency = <100000>;
};
```

```
dtc -@ -I dts -O dtb -o BB-I2C2-ORNEK.dtbo BB-I2C2-ORNEK.dts
```

BBB'nin resmi Debian image'larında overlay'ler `/boot/uEnv.txt` dosyasından yönetilir. Hazır overlay'ler `/lib/firmware/` klasöründedir:

```
# /boot/uEnv.txt (ilgili satırlar)
enable_uboot_overlays=1
uboot_overlay_addr0=/lib/firmware/BB-I2C2-00A0.dtbo
#disable_uboot_overlay_video=1      # HDMI'ı kapatıp o pinleri serbest bırakır
```

> **Not**
> Raspberry Pi'deki `config.txt` içindeki `dtoverlay=...` satırları da aynı mekanizmadır. Overlay kavramı BeagleBone'a özel değildir.

---

13. Özet

Kavram	Bir cümlede
Device Tree	Keşfedilemeyen donanımı kernel'e tarif eden, ağaç yapısındaki veri
.dts	Bir kartın insan tarafından okunabilir kaynak tarifi
.dtsi	Birden fazla `.dts` tarafından include edilen ortak (genelde SoC) tarif
dtc	DTS ↔ DTB dönüşümü yapan derleyici
.dtb	Derlenmiş binary. Bootloader RAM'e yükler, kernel okur.
.dtbo	DTB'ye sonradan uygulanan yama (cape/HAT için)
compatible	Node ile driver'ı eşleştiren string
reg	Cihazın adresi (ve memory-mapped ise boyutu)
status	`"okay"` → kullan, `"disabled"` → kullanma
label / phandle	Node'a takma ad ve başka node'dan ona referans (`&label`)
/proc/device-tree	Çalışan kernel'in gördüğü device tree

BeagleBone Black için akış:

```
am33xx.dtsi + am335x-bone*.dtsi + am335x-boneblack.dts
        │  (cpp + dtc  /  make dtbs)
        ▼
am335x-boneblack.dtb  ──►  SD/eMMC: /boot/...
        │  U-Boot: load → fdtaddr (0x88000000)
        ▼
bootz/bootm ${loadaddr} - ${fdtaddr}   (r2 = DTB adresi)
        │
        ▼
Kernel DTB'yi okur → device'ları oluşturur → compatible ile driver'ları probe eder
```

Kaynaklar

- Görseller: Bootlin - Linux kernel and driver development training slaytları ve bootlin/training-materials deposu. Lisans: CC BY-SA 3.0
- Devicetree Specification - devicetree.org
- Linux kernel dokümantasyonu: Devicetree
- Bootlin - Device Tree 101 webinar
- eLinux.org - Device Tree Usage